Computer Architecture and System Software

Lecture 02: Overview of Computer Systems & Start of Chapter 2

Instructor:
David Bosc
Applied Computer Science
University of Winnipeg

Announcements

- Reminder: No LLM/AI usage for any deliverables in this course (lab, assignment, etc.)

- Assignment 1 will be posted this week
  - Will be due Jan 30th (~2 weeks from now)
  - Some programming, mostly theory and math

Chapter 2

Representing and Manipulating Information

Information is bits + context

- Recall: all information in a system is represented as a bunch of bits.
- What does this mean ?
  1. All information has to be encoded by numbers
  2. All numbers have to be stored in binary

Decimal Number System

- We operate in a decimal system world
- For example a number 7392 represents

$$7 \times 10^3 + 3 \times 10^2 + 9 \times 10^1 + 2 \times 10^0$$

- Each coefficient is one of the ten symbols
  - Called digits
- This system is said to be of base (or radix) 10 because it uses 10 digits

Why don't Computers use Base 10?

Base 10 Number Representation

- Each coefficient is one of the ten digits {0, …, 9}
  - That is why fingers are also called digits
  - In use for thousands of years
- Implementation
  - Digits are hard to store
    - ENIAC (first electronic computer) used 10 vacuum tubes/digit
  - Digits are hard to transmit
    - High precision required to encode 10 signal levels on 1 wire
  - Messy to implement digital logic functions
    - Addition, multiplication, etc.

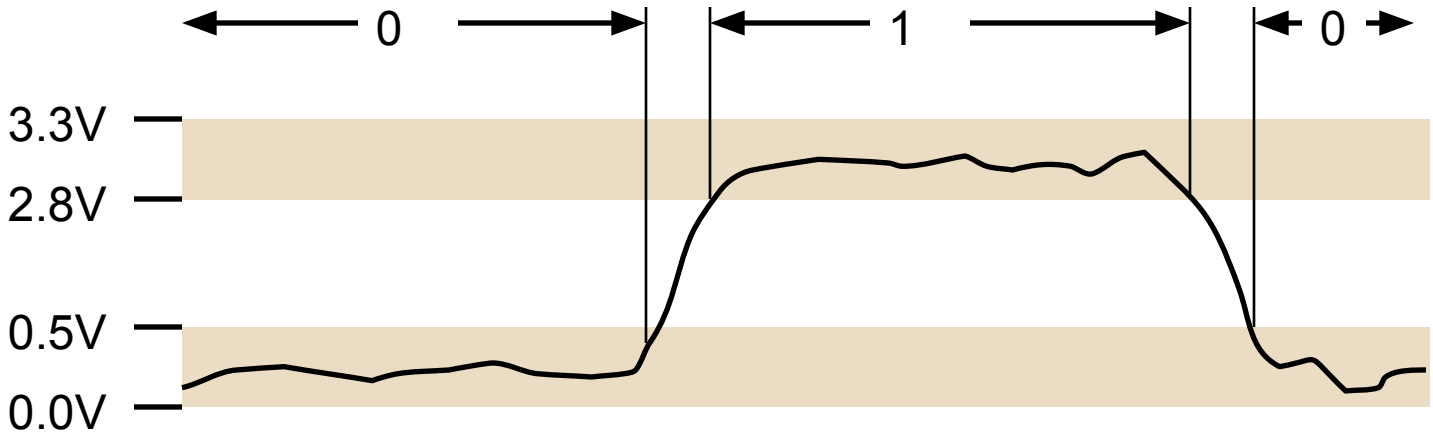
Binary Representations

Base 2 Number Representation

- Computers store and process information as bits
- Coefficients can take on only two values: 0 or 1
- Can represent more complex values if:
  - Grouped together (not useful in isolation)
  - Assigned context (interpretation)
- Represented, stored, and transmitted as:
  - Presence or absence of a hole in punch card
  - High or low voltage on a wire
  - A magnetic domain oriented clockwise or counter-clockwise

Electronic Representation

- Easy to store with bistable elements
- Reliably transmitted on noisy wires
- Straightforward implementation of arithmetic functions



Binary Number System

- In base 10, number 7392 represents

$$7 \times 10^3 + 3 \times 10^2 + 9 \times 10^1 + 2 \times 10^0$$

- In base 2, number $(10010100)_2$ represents

$$1 \times 2^7 + 0 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$$

$= 128 + 16 + 4 = (148)_{10}$

Hexadecimal Notation

- Problem: Both notations are inconvenient
  - Binary is too verbose
  - Decimal: difficult to determine binary equivalent from just looking
- Solution: Hexadecimal (base 16)
  - 10 decimal digits + 6 more

Hexadecimal Notation

- Coefficients can take on 16 different values
  - {0, …, 9, A, …, F}
- A corresponds to 10, B to 11, *etc.*
- Each coefficient can be represented by 4 bits
- Now a Byte can be represented as:
  - 00_{16} … FF_{16}
  - 0_{10} … 255_{10}
  - $0000\ 0000_2$ … $1111\ 1111_2$
- Example:
  - $0110\ 1001_2 = 69_{16}$

Hexadecimal Notation

- Most computers use blocks of eight bits as the smallest addressable unit of memory
- Called a byte
- Range:
  - 0_{10} … 255_{10}
  - $0000\ 0000_2$ … $1111\ 1111_2$
- Hex Range:
  - 00_{16} …09_{16}, $0A_{16}$, $0B_{16}$… FF_{16}

Hexadecimal Notation

- By convention, prefix a hex number with 0x or 0X
- A nibble is 4 bits of a byte
- Byte is represented by two hexadecimal digits
  - Each represented by a nibble
- Example:
  - 0000 0000
  - 0101 1001
  - 1101 1011
  - 1010 1000 1111

Conversion

- Binary to Hex
  - Separate number into nibbles
  - Zero pad left side of string
    - 101011 should be written as 0010 1011
  - Use memory or table to convert
- Hex to Binary
  - Each digit is four bits
  - Convert each nibble independently and then put it all together

Conversion

- Hex to Dec
  - Multiply coefficients by powers of 16

$$0\text{x}7\text{AF} = 7 \cdot 16^2 + 10 \cdot 16 + 15 = 1967$$

Conversion

- Dec to Hex
  - Keep dividing by 16, the remainders become the hex digits

$$
\begin{aligned}
314156 &= 19634 \cdot 16 + 12 && \text{(C)} \\
19634 &= 1227 \cdot 16 + 2 && (2) \\
1227 &= 76 \cdot 16 + 11 && \text{(B)} \\
76 &= 4 \cdot 16 + 12 && \text{(C)} \\
4 &= 0 \cdot 16 + 4 && (4)
\end{aligned}
$$

- Result 0x4CB2C
  - Note: Result is read from bottom to top
  - Unsure, check result

Conversion

- Decimal to Binary works the same way

$$87 = 43 \times 2 + 1 \qquad (1)$$
$$43 = 21 \times 2 + 1 \qquad (1)$$
$$21 = 10 \times 2 + 1 \qquad (1)$$
$$10 = 5 \times 2 + 0 \qquad (0)$$
$$5 = 2 \times 2 + 1 \qquad (1)$$
$$2 = 1 \times 2 + 0 \qquad (0)$$
$$1 = 0 \times 2 + 1 \qquad (1)$$

Result: 101 0111

WorkSheet 1

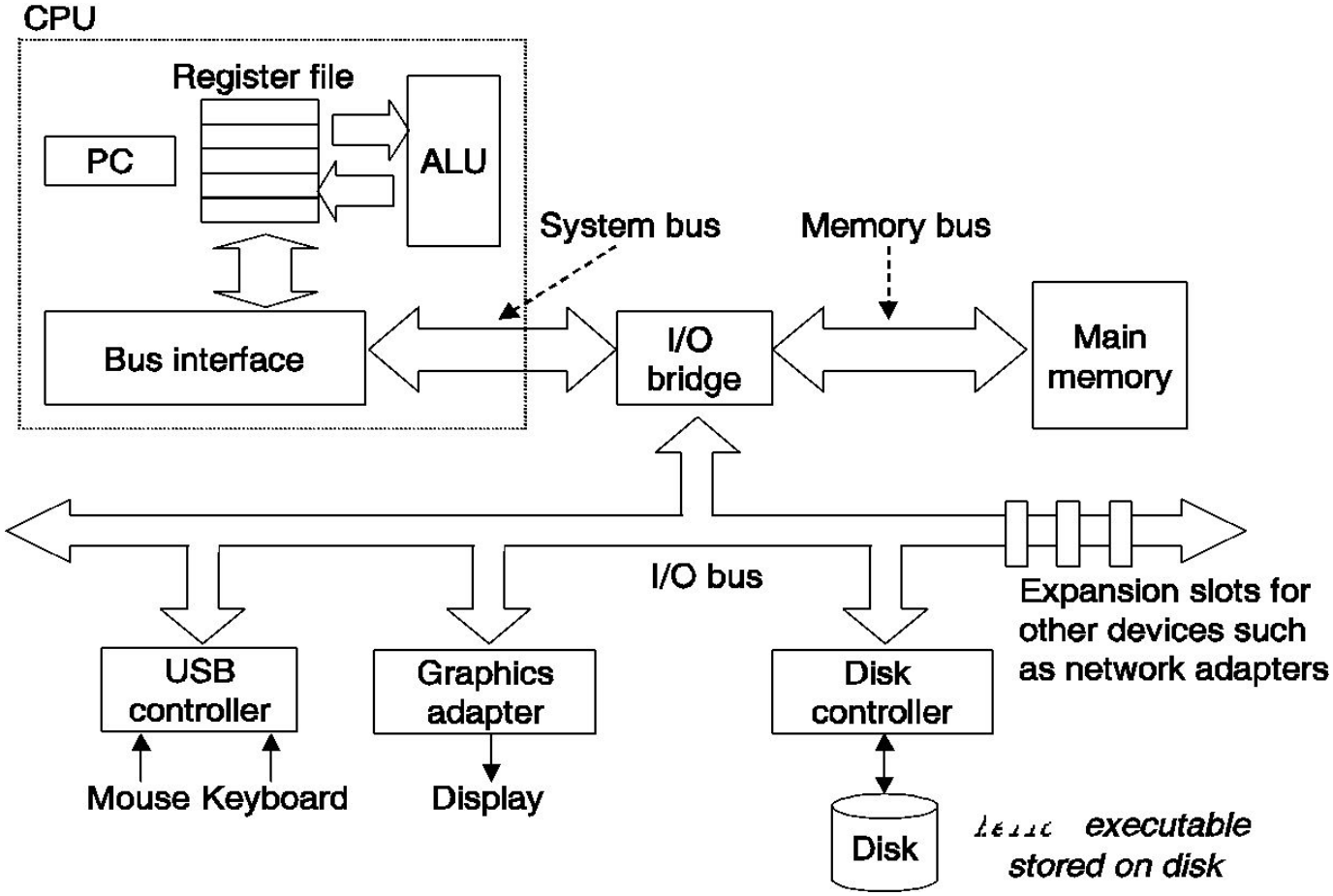
- Covers material up to here

Miscellaneous

Sections 2.1.2 – 2.1.5

From Number Systems to Addresses

- Recall:



Back to Buses

- Buses are collections of wires to shuttle data between components
  - Transfer fixed chunks of data
  - **Word size** specifies the size of "chunk" transferred
- Memory is organized as a linear array of bytes
  - *i*.e. each byte has a unique address
- Thus: the size of the address bus determines the amount of physical memory that can be addressed

Example

- 32-bit machine refers to the size of the address bus
- By definition, the memory is limited to

$$
\begin{aligned}
2^{32} &= 4,294,967,296 && \text{Addressable Bytes} \\
&\div 1,024 && \text{Bytes per KiloByte} \\
&\div 1,024 && \text{KiloBytes per MegaByte} \\
&= 4,096 \text{ MB}
\end{aligned}
$$

- 64 bit machines can address 1.8×10^{19} bytes

Units of memory

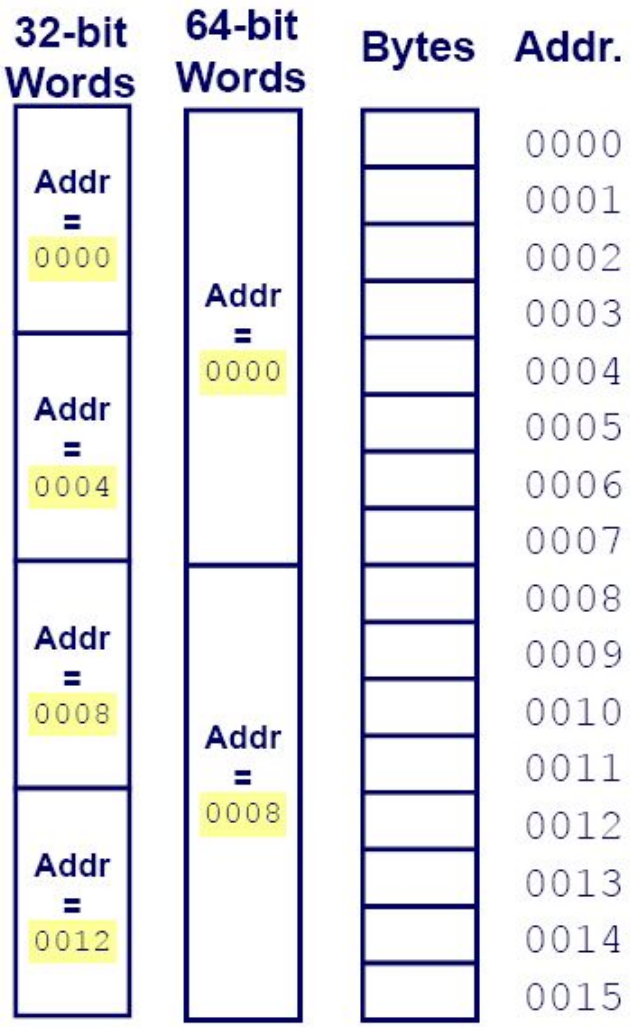
- Bit
- Nibble (4 bits)
- Byte (8 bits or 2 nibbles)
- Kilobyte (1024 bytes)
- Megabyte (1024 kilobytes)
- Gigabyte (1024 megabytes)
- Terabyte (1024 gigabytes)
- Pedabyte =1024x1024x1024x1024x1024x8 bits

Units of memory

- Note that in the International System of Units (SI), the prefix kilo means 1000.
- Sometimes you will come across texts/articles that adopt this notation, and use 1 kB to mean 1000 bytes.
- In this course, we will stick with the binary metric (1 KB = 1024 bytes, or 1 KiB)

Memory Organization

- Addresses specify byte locations
  - Address of first byte in word
  - Address of successive words differ by 4 (32-bit) or 8 (64-bit)



Aside: Data Sizes

- Programming languages and processors support multiple data formats
  - Integers: char, short, int, long
  - Pointers or references
  - Floating point (real numbers): float, double
- These data formats are different sizes
  - char = 1 byte, short = 2 bytes, int = 4 bytes (word size), float = 4 bytes, double = 8 bytes
- Have different internal representations
  - Int and float: same bit pattern represents 2 different values

Recall

- 1 byte = 8 bits
  - 2^8 different combinations possible
  - $2^8 = 256$, so we can represent numbers 0-255
  - In other words, our range is $[0,2^8-1]$

- What is our range for 2 bytes?

Recall

- 1 byte = 8 bits
  - 2^8 different combinations possible
  - $2^8 = 256$, so we can represent numbers 0-255
  - In other words, our range is 2^8-1

- Question: What is our range for 2 bytes?

Back to Memory: Addressing

- To use multi-byte objects we need two conventions
  - Object addresses
  - Byte order
- On most machines:
  - Objects are stored contiguously
  - Object's address is first byte of the object

Byte Ordering Example

Big Endian

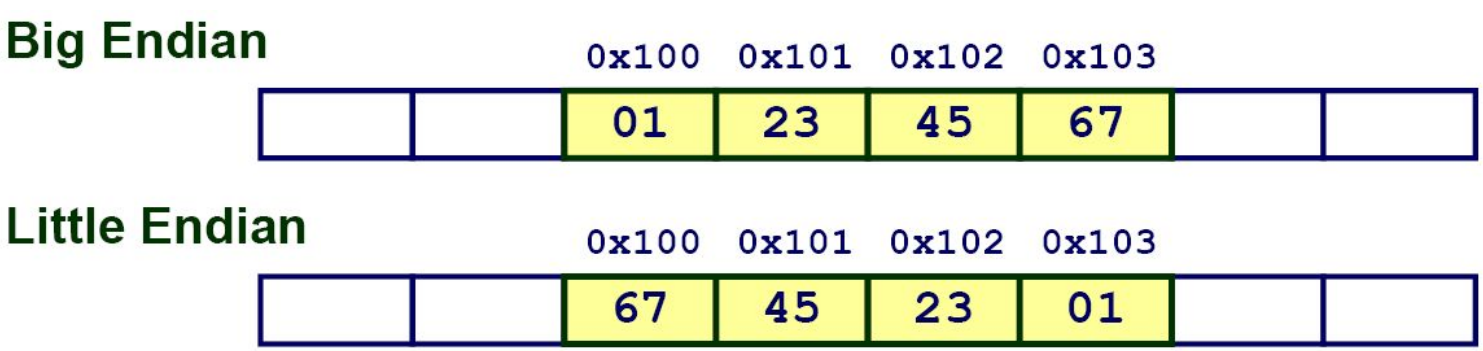
- Least significant byte has highest address

Little Endian

- Least significant byte has lowest address

Example

- Int `x` has 4-byte representation `0x01234567`
- Address given by `&x` is `0x100`



String Representation

- A sequence of characters terminated by a 0 character
- Each character is represented by a standard encoding
- Most systems use ASCII (American Standard Code for Information Interchange)
  - Wait! What?
  - Most systems *__used__* to use ASCII
  - Now, UTF-8 is the most common
    - We will use ASCII for this course
      - The first 128 characters of ASCII are the same as UTF-8

String Representation

- A sequence of characters terminated by a 0 character
- Each character is represented by a standard encoding
- Most systems use ASCII (American Standard Code for Information Interchange)
- Example string “Hello” has the encoding

  0x48 0x65 0x6C 0x6C 0x6F 0x00
- Looks the same on any system using ASCII

Bit-Level Operations

Section 2.1.7

Boolean Logic

- To manipulate data, computers change the value of bits based on other bits
- A bit naturally encodes the logic values:
  - True (1)
  - False (0)
- Bit manipulation is based on logic

Boolean Logic

- Developed by George Boole in 19th Century
- Given Boolean variables p & q
- NOT ~p: yields true if p is false
- AND p & q: yields true if both p and q are true
- OR: p | q: yields true if either p or q are true
- XOR p ^ q: yields true if only one of p or q is true

Bit-Level Operators

- Boolean logic can be applied to the integral data types:
  - long, int, short, char
- View arguments as bit vectors
- Arguments applied bit-wise
- Examples (Char data type)
  - `~0x41 □ 0xBE`
  - `~0x00 □ 0xFF`
  - `0x69 & 0x55 □ 0x41`
  - `0x69 | 0x55 □ 0x7D`
- These operators perform the same in C, C++, Java

Bit-Level Operators

- Boolean logic can be applied to the integral data types:
  - long, int, short, char
- View arguments as bit vectors
- Arguments applied bit-wise
- Examples (Char data type)
  - `~0x41 □ 0xBE` □ Using ASCII, this statement
  - `~0x00 □ 0xFF` is equivalent to (~A = ┘)
  - `0x69 & 0x55 □ 0x41`
  - `0x69 | 0x55 □ 0x7D`
- These operators perform the same in C, C++, Java

Bit-Level Operators

- Although the smallest addressable unit of memory is a byte, we can probe specific bits using bit-level operators.

- Example:

```
   1010 0100
 & 0000 0001 = 0

   1010 0101
 & 0000 0001 = 1
```

In general, (X&1) returns the lowest order bit of X.

Bit-Level Operators

- The last trick is a fast way of determining whether a binary number is even or odd. (Can you see why?)
- Bit-level operations are simple operations that take few cycles on a CPU
- You should substitute more complicated math operators (i.e. dividing) with bit level operators wherever possible

Example

Q12) What does the following Java function do? Give an example of how it could be used. (3 marks)

```
public static int unknown_Function (int x, int n) {
     //n can only be in the interval [0,3]
     return 0xff & (x >> (n << 3));
}
```

Logical Operators

Section 2.1.8

Logical Operators

- C, C++, and Java also have logical operators
  - Not: !
  - And: &&
  - Or: ||
- What's the difference ?
  - Used when evaluating expressions to obtain a single **relational** result
  - Input/Output always interpreted as 1 or 0

Examples

- int a = 5, b =10, c = 15;
- (a<b) && (a<c) = 1
- (a<b) && (c<a) = 0

- More in text on pg. 54 (56 in 2nd ed)

Shift Operators

Section 2.1.9

Shift Operations

- C, C++, and Java also provide shift operations
- Shift the bit vector a specified number of bits left or right
- These operations are left shift, right logical shift, and right arithmetic shift

Shift Operations

- Left shift
  - Denoted x << i
- Right logical shift
  - Fills the left end with zeros
  - Denoted x >> i in C, C++
    - Unsigned data must use logical shift
  - Denoted x >>> i in Java
- Right arithmetic shift
  - Fills the left with repetitions of the sign bit (left-most bit)
  - Denoted x >> I
    - C,C++ arithmetic shift is used on signed data

Examples

Argument `x`	`01100010`
`<< 3`	`00010`**`000`**
Log. `>> 2`	**`00`**`011000`
Arith. `>> 2`	**`00`**`011000`

Argument `x`	`10100010`
`<< 3`	`00010`**`000`**
Log. `>> 2`	**`00`**`101000`
Arith. `>> 2`	**`11`**`101000`

Examples

Argument `x`	`01100010`
`<<  3`	`00010`**`000`**
Log. `>>  2`	**`00`**`011000`
Arith. `>>  2`	**`00`**`011000`

Argument `x`	`10100010`
`<<  3`	`00010`**`000`**
Log. `>>  2`	**`00`**`101000`
Arith. `>>  2`	**`11`**`101000`

Worksheet 2

- Covers everything up until here

Chapter 2

Section 2.2 Integer Representations

Integer Data Types

Motivation:

- Important to understand the representation of the data types you are working with.
  - Integral types represent a finite range of integers
- Sizes:
  - 1, 2, 4, or 8* bytes
- Unsigned: Only non-negative integers (the representation we've been using so far)
- Signed: Positive and negative integers
  - Java only supports signed numbers
- http://en.wikipedia.org/wiki/Comparison_of_basic_instructions_of_programming_languages#Integers

Integer Data Types

- Size determines range of integers a variable can represent
  - Each pattern represents a single integer
- How many different bit patterns can a w-length bit vector have ?
  - 1 bit = 2 patterns (0 or 1)
  - 4 bits = 2^4 = 16 patterns
  - 8 bits = 2^8 = 256 patterns
  - 16 bits = 2^{16} = 65536 patterns

Integer Data Types

- Hence, # of patterns equal # of integers

Size (bytes)	Num. of Patterns	Unsigned Range	Signed Range
1	256	$[0 \ldots 255]$	$[-128 \ldots 127]$
2	$65,536$	$[0 \ldots 65,535]$	$[-32,768 \ldots 32,767]$
4	$4,294,967,296$	$[0 \ldots 4,294,967,295]$	$[-2,147,483,648 \ldots 2,147,483,647]$
8	$\approx 1.8 \times 10^{19}$	$\approx [0 \ldots 1.8 \times 10^{19}]$	$\approx [-9.2 \times 10^{18} \ldots 9.2 \times 10^{18}]$

- Question: How are these integers represented ?

Unsigned Integers

- Assume we have an integer data type of w bits.
- Denote an entire vector as $\vec{x}$
- Denote the individual bits in a vector as $[x_{w-1}, x_{w-2}, \ldots, x_0]$
- Treating $\vec{x}$ as a number written in binary notation, we obtain the *unsigned* representation of $\vec{x}$

A function $B2U_w$ (binary to unsigned) is given as

$$B2U_w(\vec{x}) = \sum_{i=0}^{w-1} x_i 2^i$$

Observations

- Equation formalizes conversion of binary numbers discussed last class
- Unsigned minimum, $Umin_w$, is always 0
- Unsigned maximum, $UMax_w = \sum_{i=0}^{w-1} 2^i = 2^w - 1$

Signed Integers

- Many applications require negative values
- Two's complement form: Most common representation of signed numbers
- Most significant bit has negative weight

$$B2T_w(\vec{x}) = -x_{w-1}2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i$$

- Most significant bit is also called the sign bit
  - Set to 1 $\rightarrow$ number is negative
  - Set to 0 $\rightarrow$ number is positive

Example

X		$B2U_4$	$B2T_4$
Hex	Binary		
A	1010		
0	0000		
3	0011		
B	1011		
C	1100		
F	1111		

Example

X		$B2U_4$	$B2T_4$
Hex	Binary		
A	1010	10	
0	0000	0	
3	0011	3	
B	1011	11	
C	1100	12	
F	1111	15	

Example

X		$B2U_4$	$B2T_4$
Hex	Binary		
A	1010	10	-6
0	0000	0	0
3	0011	3	3
B	1011	11	-5
C	1100	12	-4
F	1111	15	-1

Observations

- Signed minimum, $TMin_w = [10 \cdots 0] = -2^{w-1}$
- Signed maximum, $TMax_w = [01 \cdots 1] = \sum_{i=0}^{w-2} 2^i = 2^{w-1} - 1$
- Two's complement range is asymmetric: $|TMin_w| = |TMax_w| + 1$.
- $UMax_w = 2TMax_w + 1$
  - Two's complement notation reserves half of the bit patterns to represent negative values
- -1 has the same bit representation as $UMax_w$ (a string of all 1s)
- 0 is the same in both representations

Other Representations

Ones' Complement: This is the same as two's-complement, except that the most significant bit has weight $-(2^{w-1}-1)$ rather than -2^{w-1}.

$$B2O_w(\vec{x}) = -x_{w-1}(2^{w-1}) + \sum_{i=0}^{w-2} x_i 2^i$$

Sign-Magnitude: The most significant bit is a sign bit that determines whether the remaining bits should be given negative or positive wieght.

$$B2S_w(\vec{x}) = (-1)^{x_{w-1}} \cdot \left(\sum_{i=0}^{w-2} x_i 2^i \right)$$

Unsigned & Signed Numeric Values

X	B2U(*X*)	B2T(*X*)
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	–8
1001	9	–7
1010	10	–6
1011	11	–5
1100	12	–4
1101	13	–3
1110	14	–2
1111	15	–1

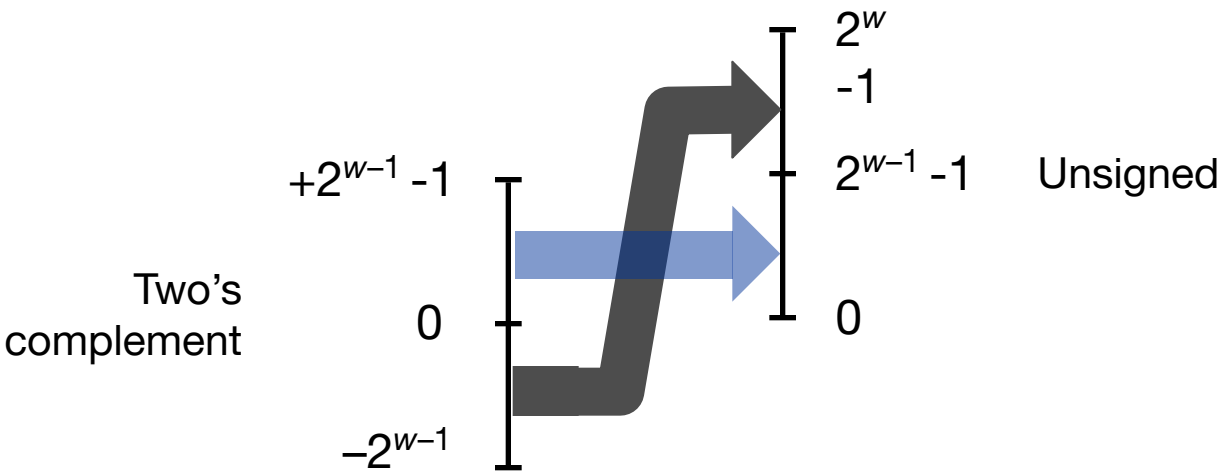
- Equivalence
  - Same encodings for nonnegative values
- Uniqueness
  - Every bit pattern represents unique integer value
  - Each representable integer has unique bit encoding
- Can Invert Mappings

Conversion

- What happens if you interpret a signed integer as an unsigned one, or *vice versa* ?
- Bit pattern is the the same, but the bits may be interpreted differently

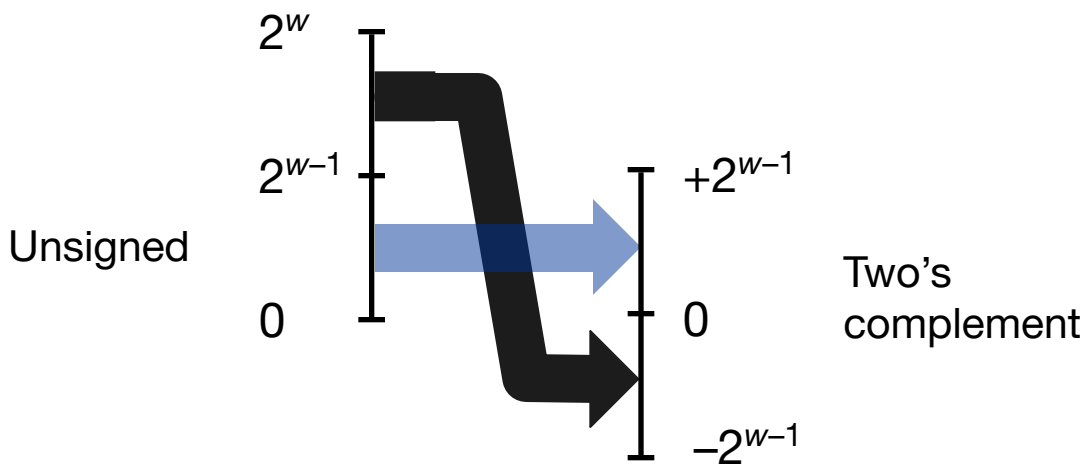
Two's Compliment 2 Unsigned

- Use the fact that they have identical bit representations to derive a numerical relationship



Unsigned 2 Two's Complement

- Same process as previous slide



Shortcut conversion

- To quickly read a **negative** signed number:
  - If the binary number starts with 1, we know it's a negative number
  - Flip all bits and add 1 to the result
  - Read as (-) [unsigned number]

- 4-bit example:

  Signed 1111

  Flip all and add 1 □ 0000 + 1 = 0001 = 1

  Therefore, 1111 = - (1)

Another example

- Signed 1100
  - Flip all bits 0011 = 3
  - Add 1 = 4
  - Therefore, 1100 = -4

Worksheet 3

- Questions 1-3 cover up to here

Expanding Bit Representation

- Common operation: convert from smaller to larger data type
  - e.g. converting an short to an int, or an int to a long
- Should always be possible
- Unsigned: add leading zeros (*i.e.* zero extension)
- Signed: repeat sign bit (*i.e.* sign extension)
- Example:
  - Unsigned 4 bit □ 8 bit: 1001 □ 0000 1001
  - Signed 4 bit □ 8 bit: 1001 □ 1111 1001

Truncating Bit Representation

- What happens when you want to reduce the number of bits representing a number
- When truncating a w-bit number to a k-bit number, we drop the high-order w-k bits
- Truncating a number can alter its value
  - A form of overflow
  - e.g. 0x00FF □ 0xFF changes from positive to negative with the two's complement representation
- Question: What is the value of the integer after truncation ?

Aside: Modulus After Division

- Mod(x,y) or x % y: The amount by which *x* exceeds the largest integer multiple of *y* that is not greater than *x*, *i.e.* $x - ny$, where $n = \text{floor}(x/y)$
  - 20 % 3 = 2
    - (20 = 3*6 + 2) □ Mod gives us the remainder
  - 45 % 7 = 3
    - (45 = 7*6 + 3) □ Mod gives us the remainder
  - -45 % 7 = 4
    - (-45 = -7*7 + 4)

  - NOTE: Remainder is always positive!

Truncating Continued

The effects of truncation are as follows:

- w: Number of bits before truncation
- k: Number of bits after truncation

$$
\begin{aligned}
B2U_k([x_k, x_{k-1}, \ldots, x_0]) &= B2U_w([x_{w-1}, x_{w-2}, \ldots, x_0]) \mod 2^k \\
B2T_k([x_k, x_{k-1}, \ldots, x_0]) &= U2T_k(B2T_w([x_{w-1}, x_{w-2}, \ldots, x_0]) \mod 2^k)
\end{aligned}
$$

Chapter 2

2.3 Integer Arithmetic

Unsigned Addition

- Just like base 10 addition
- Except we perform addition base 2
- Example:

$$
\begin{array}{r}
1 \leftarrow \textit{Carry the one} \\
110 \\
\underline{+101} \\
1011
\end{array}
$$

Unsigned Addition

- Another Example:

$$
\begin{array}{r}
111 \\
11100 \\
\underline{+10101} \\
110001
\end{array}
\qquad \leftarrow \textit{Carry bits}
$$

- Note: We started with 5 bits and ended with 6

Unsigned Subtraction

- Example:

$$
\begin{array}{rl}
1 & \leftarrow \textit{Carry bits} \\
1 & \\
11000 & \\
-10100 & \\
\hline
00100 &
\end{array}
$$

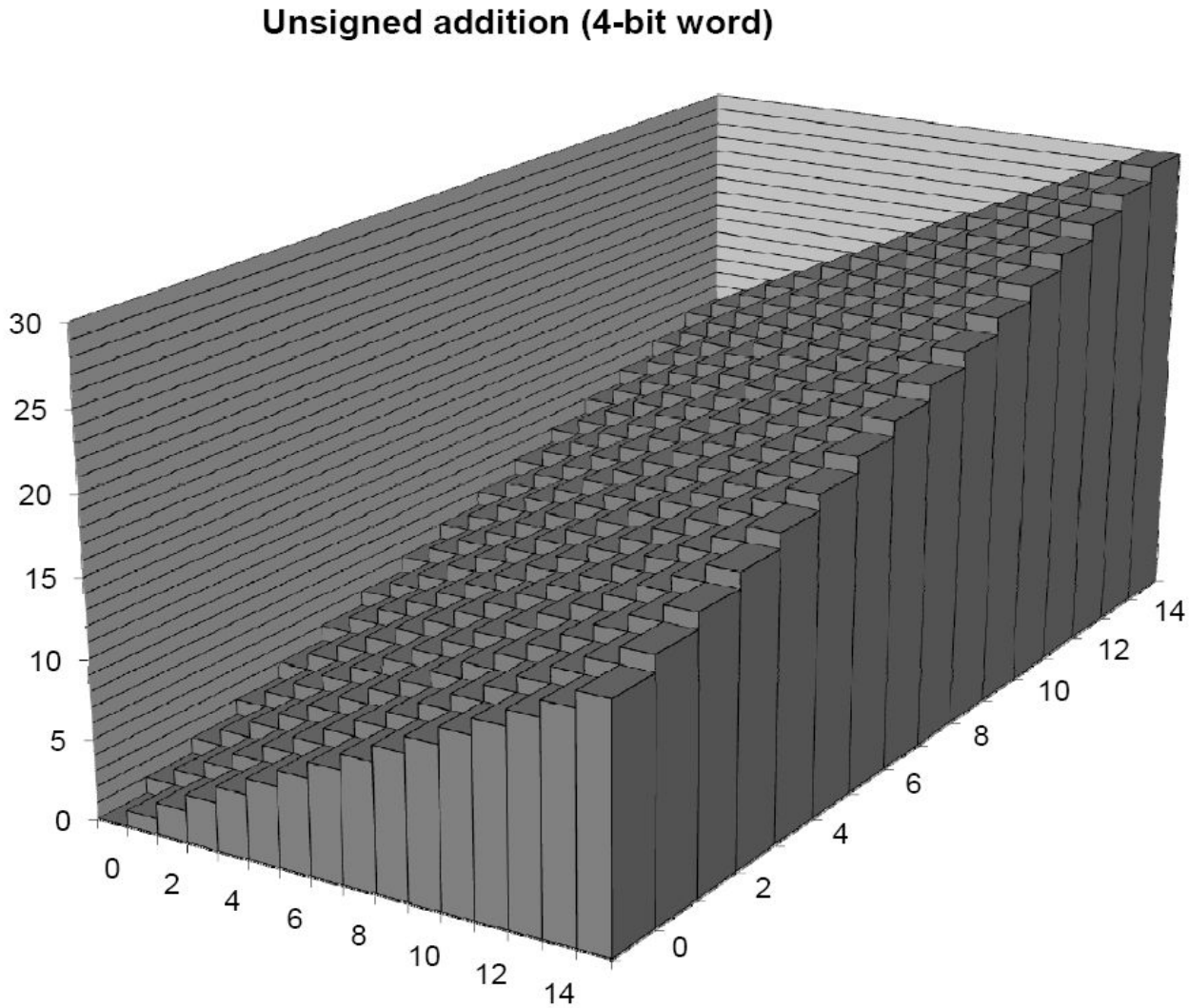
Overflow

Consider two nonnegative integers x and y:

- $0 \leq x, y \leq 2^w - 1$
- Each number can be represented by w-bit unsigned numbers
- Problem: $x + y$ has a possible range of $0 \leq x + y \leq 2^{w+1} - 2$

Example

- Arguments: 0 – 15
- Sum: 0 – 30



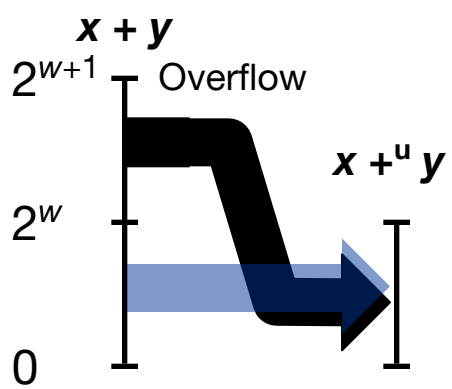
Overflow Continued

- Result of adding two w-bit integers could require w+1 bits
- Solution: Fixed precision arithmetic
- Unsigned arithmetic can be viewed as a form of modular arithmetic
- Equivalent to computing sum modulo 2^w
- Sum modulo 2^w can be obtained by discarding the high order bit in w + 1 result
- Examples:

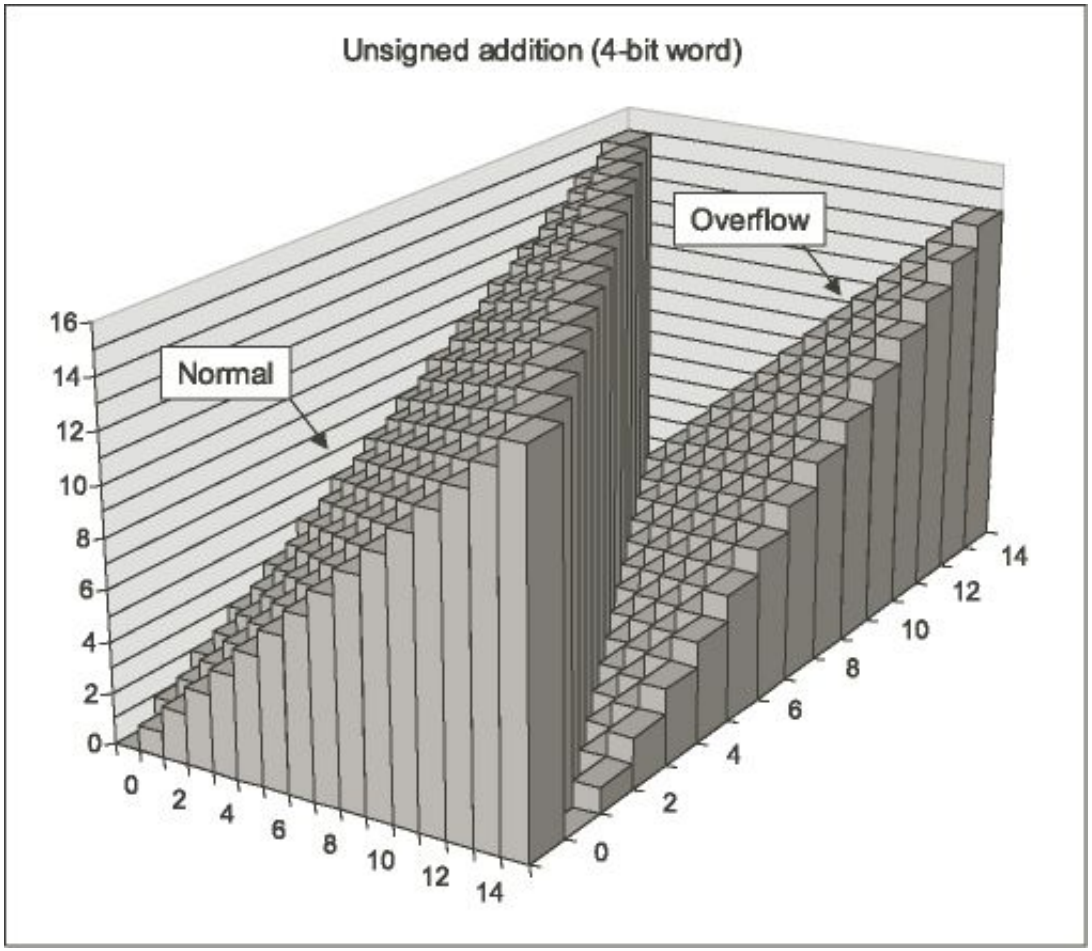
Overflow Continued

Consider two nonnegative integers x and y:

- No problem if $x + y < 2^w$
- Overflow if $2^w \leq x + y < 2^{w+1}$ and bit $w + 1$ is discarded
- Same as performing $x + y \mod 2^w$



Example


Unsigned addition (4-bit word)
Overflow
Normal
16
14
12
10
8
6
4
2
0
0
2
4
6
8
10
12
14
0
2
4
6
8
10
12
14

Two's Complement Addition

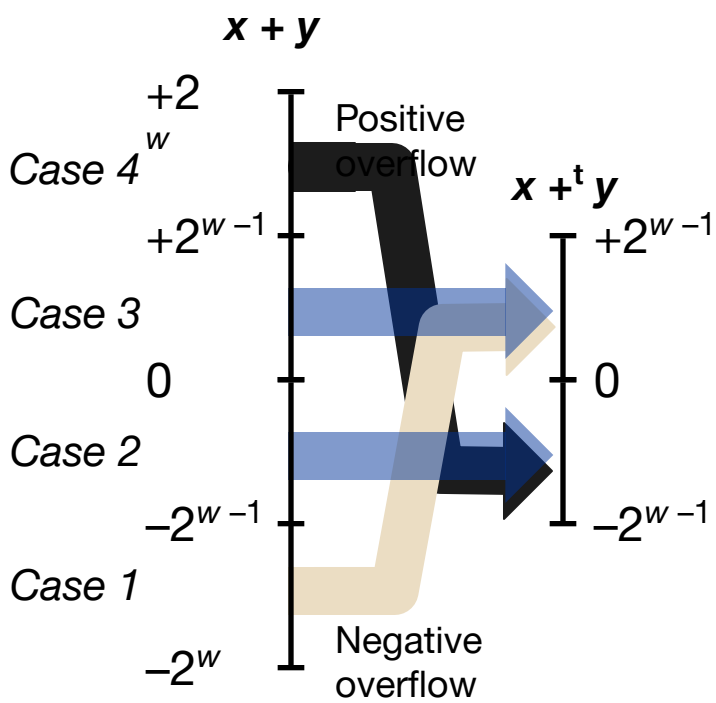
- In binary, add bit patterns as if they were unsigned
  - Again, discard high order w+1 bit
  - Interpret result as signed (remember bits + context)
  - Most computers use the same machine instruction to perform either unsigned or signed addition
- In decimal, perform the following addition
  - $x + y = U2T_w[(x+y) \bmod 2^w]$

Examples

- 4-bit examples:
  - -8 + (-5)
  - -8 + 5
  - 2 + 5
  - 5 + 5

Overflow

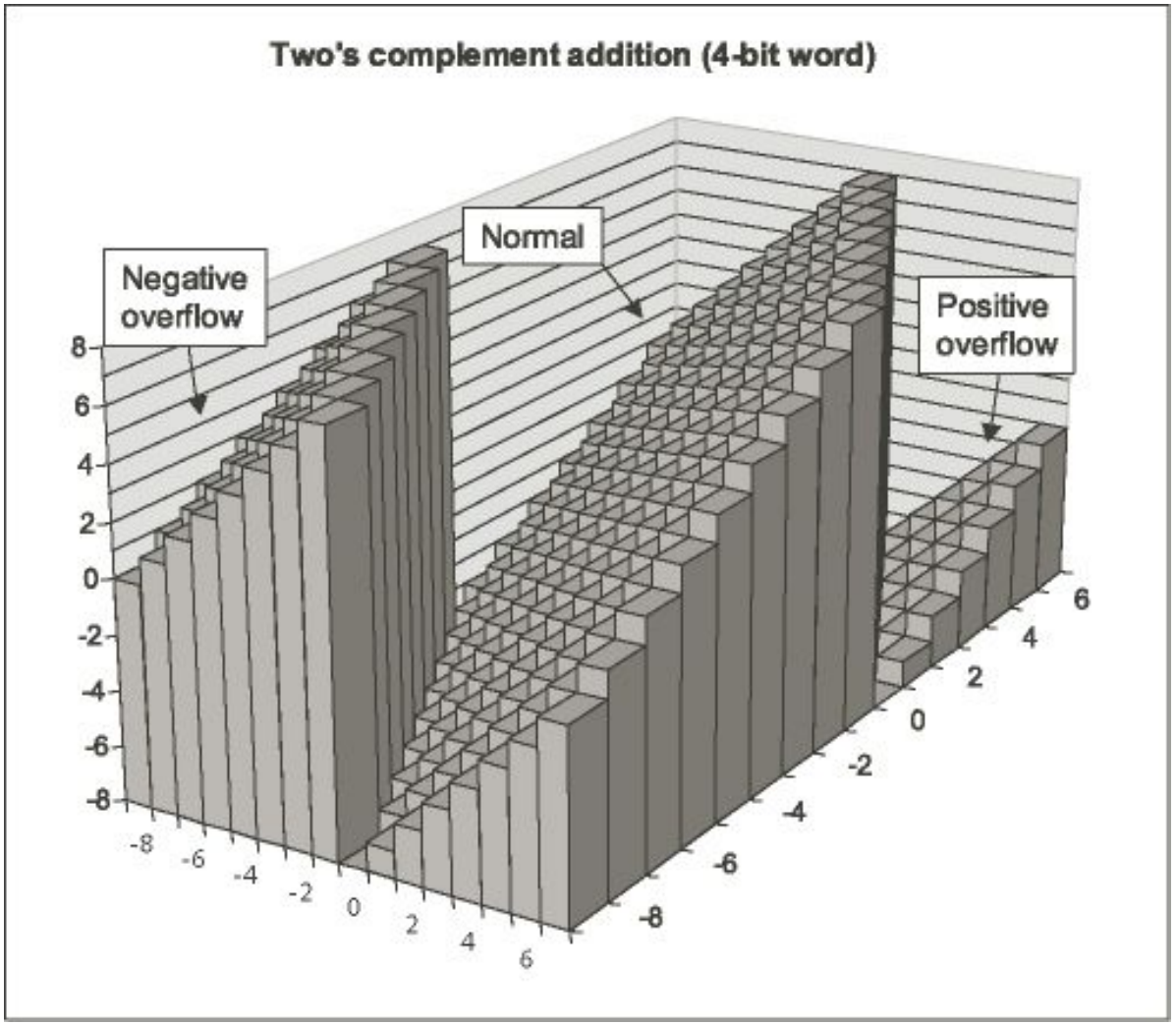
Note: In two's-compliment there are two types of overflow, positive and negative



x	y	Case
-8	-5	1
-8	-8	1
-8	5	2
2	5	3
5	5	4

4-bit addition

Example


Two's complement addition (4-bit word)
Negative overflow
Normal
Positive overflow
8
6
4
2
0
-2
-4
-6
-8
-8
-6
-4
-2
0
2
4
6
-8
-6
-4
-2
0
2
4
6

Integer Subtraction

- Subtraction is performed using addition of signed numbers
  - a – b = a + (-b)
- Example (6 bit length)
  - 25 – 5 = 25 + (-5) = 20

```
     01 1001  (25)
+    11 1011  (-5)
----------------
   101 0100   □ Truncation □ 01 0100 = 20
```

Unsigned Multiplication

- Just like multiplication in base 10
- What is the range of the product of x and y ?
- To deal with overflow we perform $(x * y) \bmod 2^w$
- Example
  - Base 2: 0110 * 1011 = 0010
  - Base 10: $(6 * 11) \bmod 2^4 = 2$

```
    0110
*   1011
-------
    0110
   01100
 0110000
-------
 1000010
```

Signed Multiplication

- Due to overflow results will also be truncated
- As a result, unsigned multiplication is the same as signed multiplication
  - *i*.e. $U2T_w((x * y) \bmod 2^w)$

Mode	x	y	x * y	Truncated
Unsigned	5	3	15	7
Signed	-3	3	-9	-1
Unsigned	4	7	28	4
Signed	-4	-1	4	-4
Unsigned	3	3	9	1
Signed	3	3	9	1

(3-bit multiplication)

Mult. and Div. by Shifting

- The cost of multiplication and division
  - Multiplication and division are expensive operations
  - Addition takes 1 to 2 cycles
  - Multiplication can take 5 to 10 cycles
  - Division 20 to 39 cycles

- Does it always have to be expensive?

Multiplying by Powers of Two

- Multiplication by 2 is the same as left shift
- Multiplication by 2^k is the same as left shift of k bits
- Examples:
  - 7 * 2
    - Before: 0000 0111
    - After: 0000 1110
  - 7 * 8
    - Before: 0000 0111
    - After: 0011 1000

Dividing by powers of 2

- Unsigned numbers
  - Same as a logical shift right
  - Ex: 42/2 = 21
    - Before: 0010 1010
    - After: 0001 0101
- Why does it work ?
  - Integer division rounds toward 0
- Divide by 2^k is a right shift of k bits
  - 42 / 8 = 5
    - Before: 0010 1010
    - After: 0000 0101

Dividing by Powers of 2

- Signed numbers
  - Use arithmetic right shift to protect the sign bit
  - Problem: integer division should round upwards for negative numbers
  - e.g. -5/2 = -2 not -3
- Need to bias value before shifting
- If $x < 0$, add $2^k - 1$ to x before right shifting where k equals the number of bits shifted
- Example:

$-5 >> 1 = (1011 + 2^1 - 1) >> 1 = (1100 >> 1) = -2$

$-5 >> 2 = (1011 + 2^2 - 1) >> 2 = (1110 >> 2) = -1$

Divide by Powers of 2

- Another example (5-bit)
  - We want -11/2 = -5 (remember we round towards 0)
  - -11 = 1 0101
  - Bias is 2^1-1 since we only want to divide by 2
  - Perform shift: (1 0101 + 2^1 – 1) >> 1

    =(1 0110) >> 1

    =(1 1011) = -5

Recall from last class

- An int is not an integer
- Example 1) Is $x^2 >= 0$?
  - float: yes
  - int: no
    - 40000 * 40000 = 1600000000
    - 50000 * 50000 = ?
- Now we can predict value above. Let's try.

Recall from last class

- What is the range of our integer?
  (Hint: We're using Java.)

Recall from last class

- What is the range of our integer? (Hint: We're using Java.)

- Java only supports signed numbers. An int data type is 4 bytes large. Therefore our range is

$$[-2^{31}, 2^{31} - 1]$$

[1000 0000 0000 0000 0000 0000 0000 0000, 0111 1111 1111 1111 1111 1111 1111 1111]

$$\approx [-2.147 \times 10^9, 2.147 \times 10^9]$$

Recall from last class

- We want to multiply 50,000 x 50,000 = 2.5×10^9

[1001 0101 0000 0010 1111 1001 0000 0000]

- But our max range = 2.147×10^9

[0111 1111 1111 1111 1111 1111 1111 1111]

- What is the predicted result after overflow?
  - How do we figure out what the result will be (a priori)?

Recall from last class

- We want to multiply 50,000 x 50,000 = 2.5×10^9

[1001 0101 0000 0010 1111 1001 0000 0000]

- But our max range = 2.147×10^9

[0111 1111 1111 1111 1111 1111 1111 1111]

- What is the predicted result after overflow?
  - How do we figure out what the result will be (a priori)?
  - Need to get rid of "sign bit" in our result
  - $2.5 \times 10^9 - 2^{31} = 0.353 \times 10^9$
  - Is this our answer?

Recall from last class

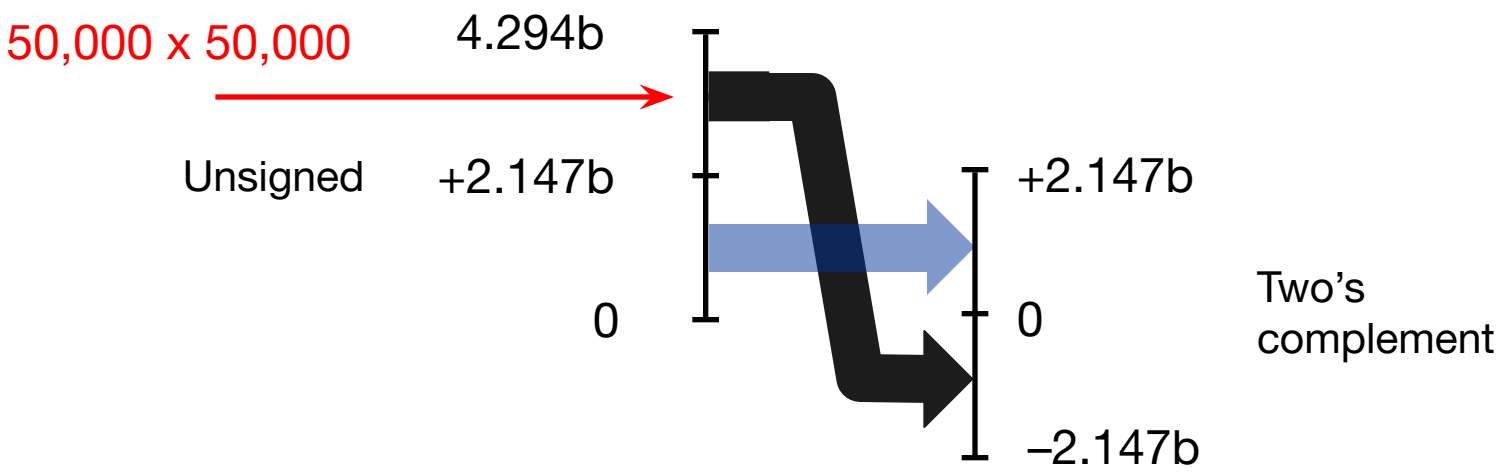
- We want to multiply 50,000 x 50,000 = 2.5×10^9

[1001 0101 0000 0010 1111 1001 0000 0000]

- But our max range = 2.147×10^9

[0111 1111 1111 1111 1111 1111 1111 1111]

- What is the predicted result after overflow?
  - How do we figure out what the result will be (a priori)?
  - Need to get rid of "sign bit" in our result
  - $2.5 \times 10^9 - 2^{31} = 0.353 \times 10^9$
  - Is this our answer?
    - No ☹ This is the value that is added to the negative power of two for the sign bit
    - $-2^{31} + 0.353 \times 10^9 = -1.794 \times 10^9$

Recall from last class

- Result falls outside of two's complement number line. It is assigned a negative value instead.